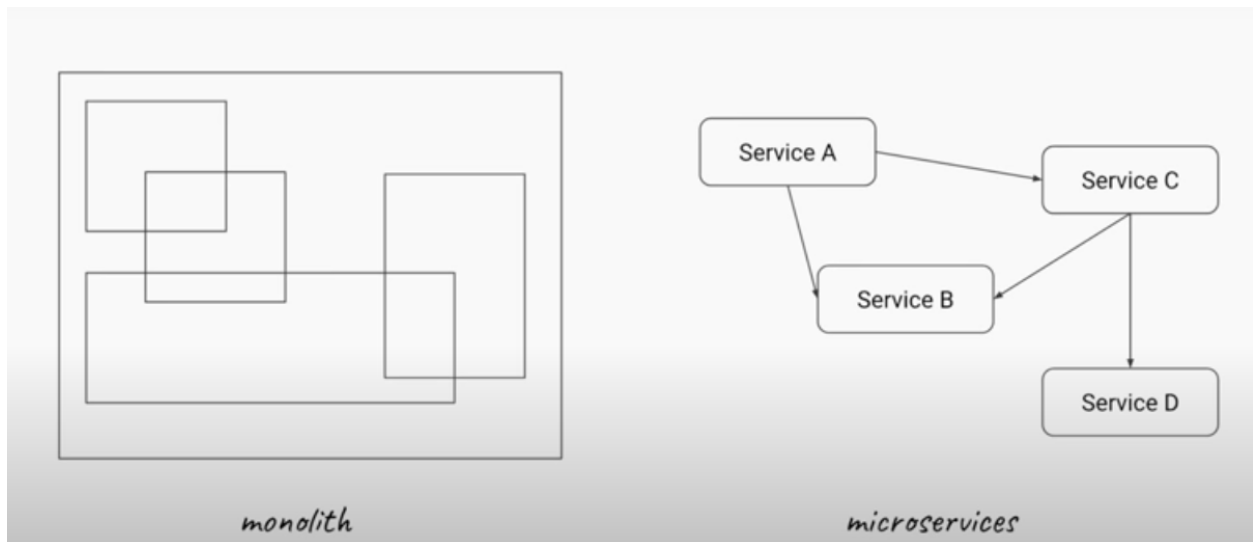


Service Mesh & ISTIO

Monolithic to Microservice



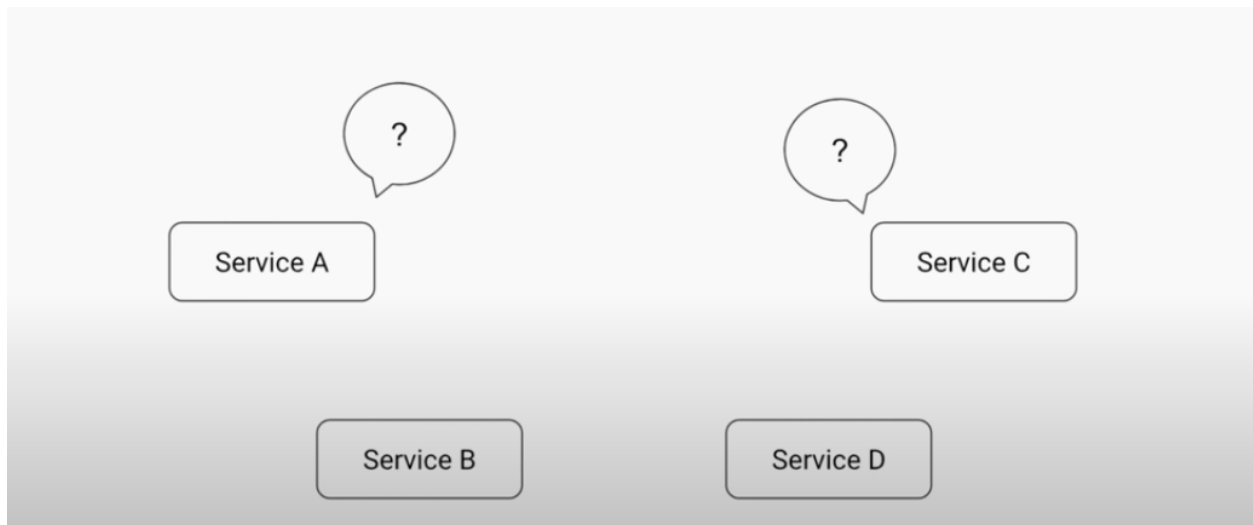
Benefits

- Faster delivery: Smaller size encourages frequent releases with less risk.
- Isolation: Single microservice cannot crash entire system.
- Scaling: We can scale individual services based on their use.
- Culture: Well defined ownership, high trust team.
- Flexibility: Polygot services, own persistence systems etc.

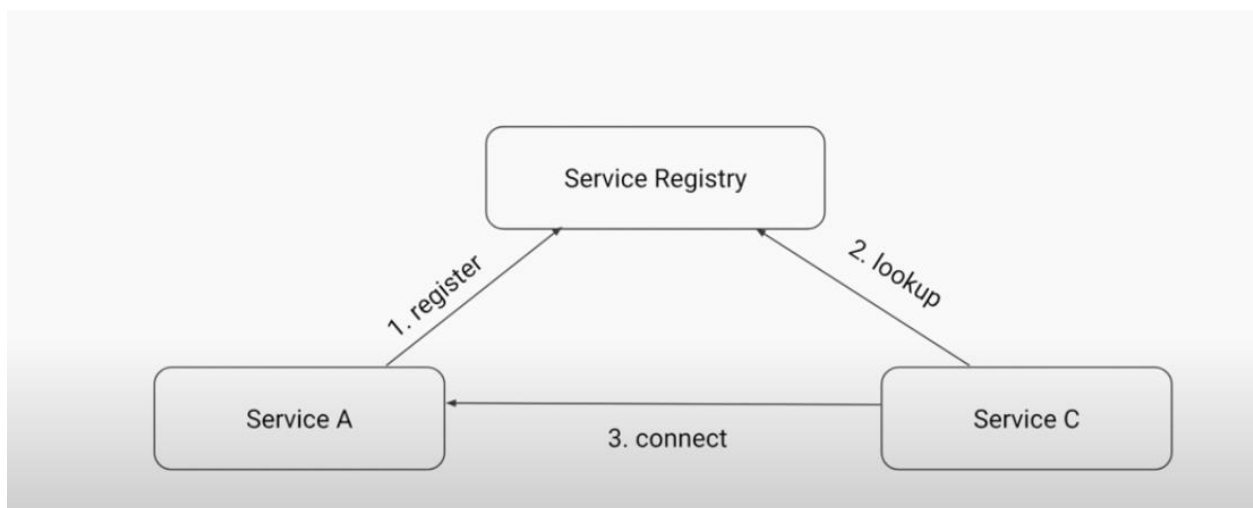
Challenges of a microservice architecture?

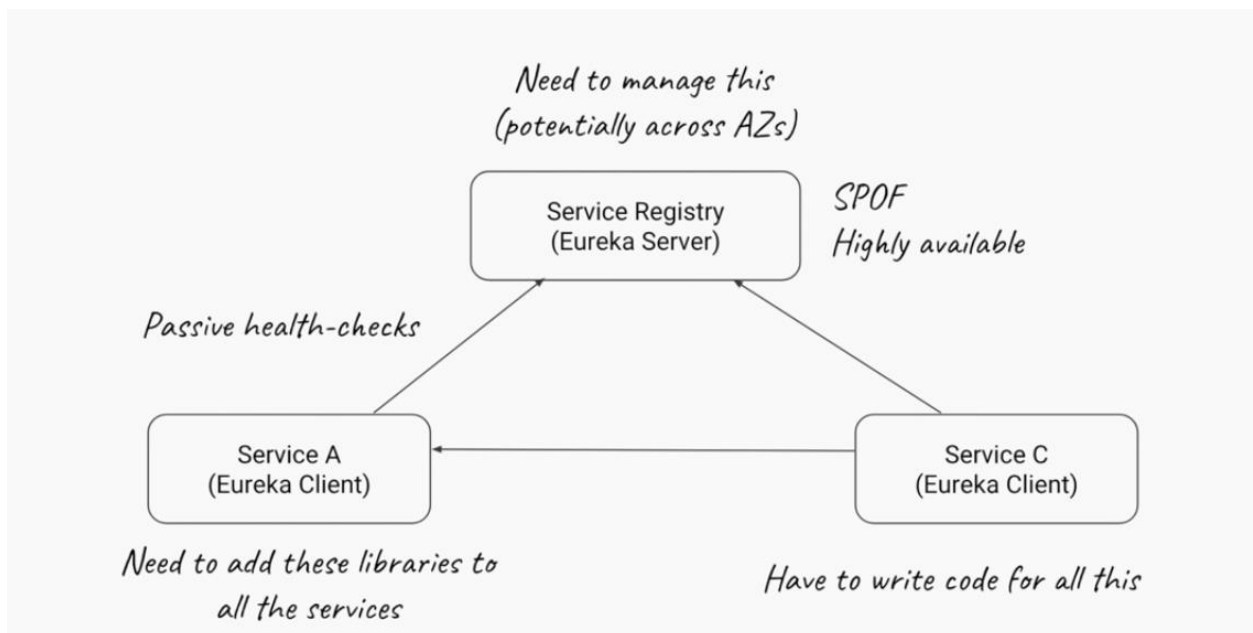
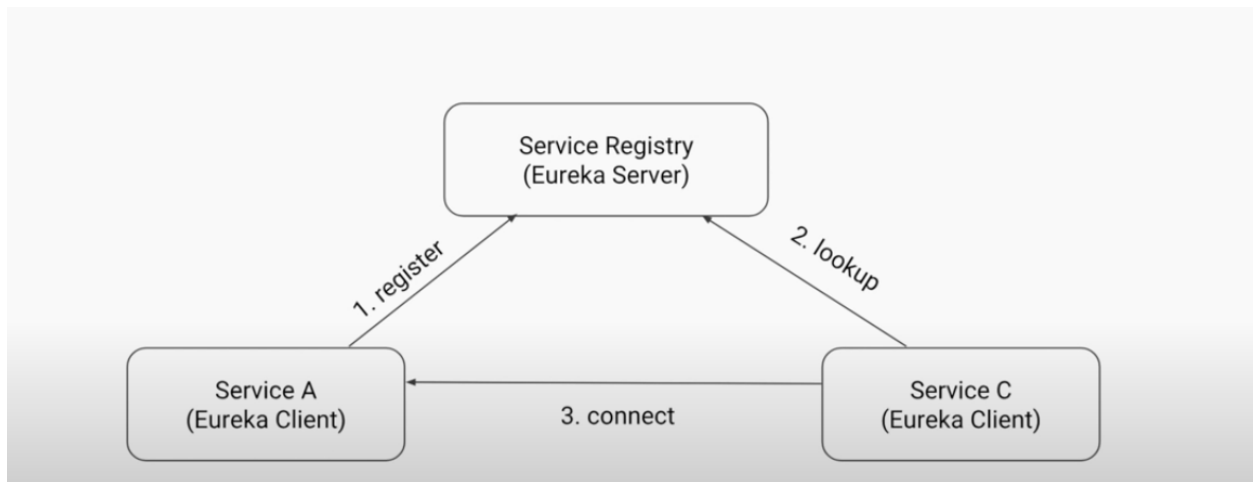
- | | |
|--------------------------------------|------------------------------|
| • Service Discovery | • Granularity |
| • Load Balancing | • Bounded Contexts |
| • Fault tolerance | • Data modelling |
| • Distributed Tracing | • Independently releasable |
| • Telemetrys | • Service contracts |
| • Security (mTLS, policies, patches) | • Smart services, dumb-pipes |

Service Discovery

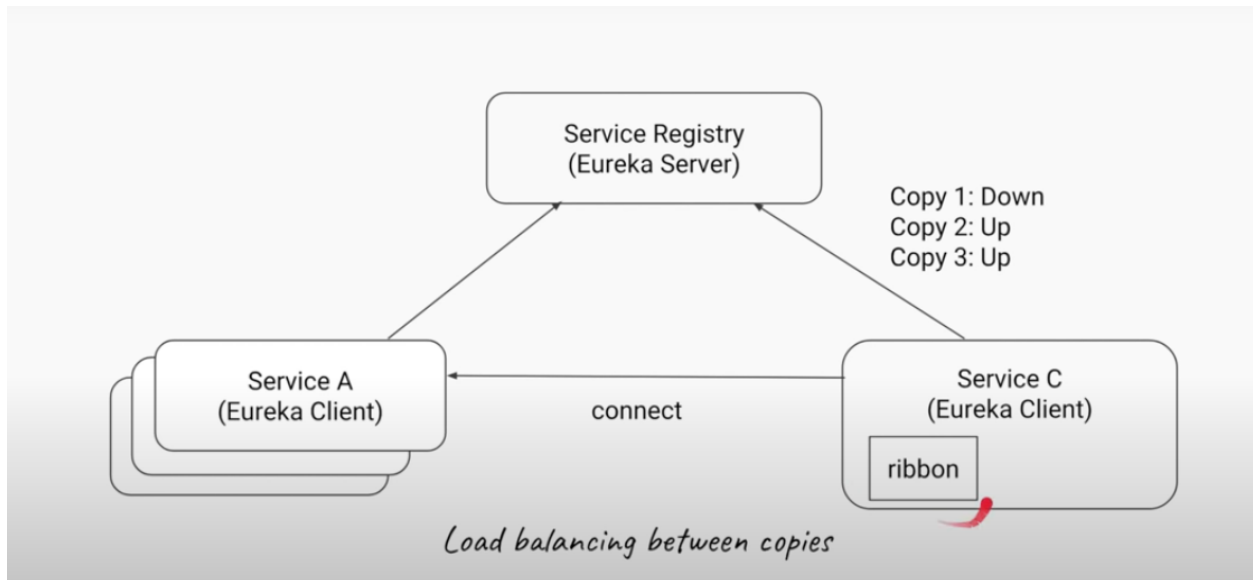


Eureka

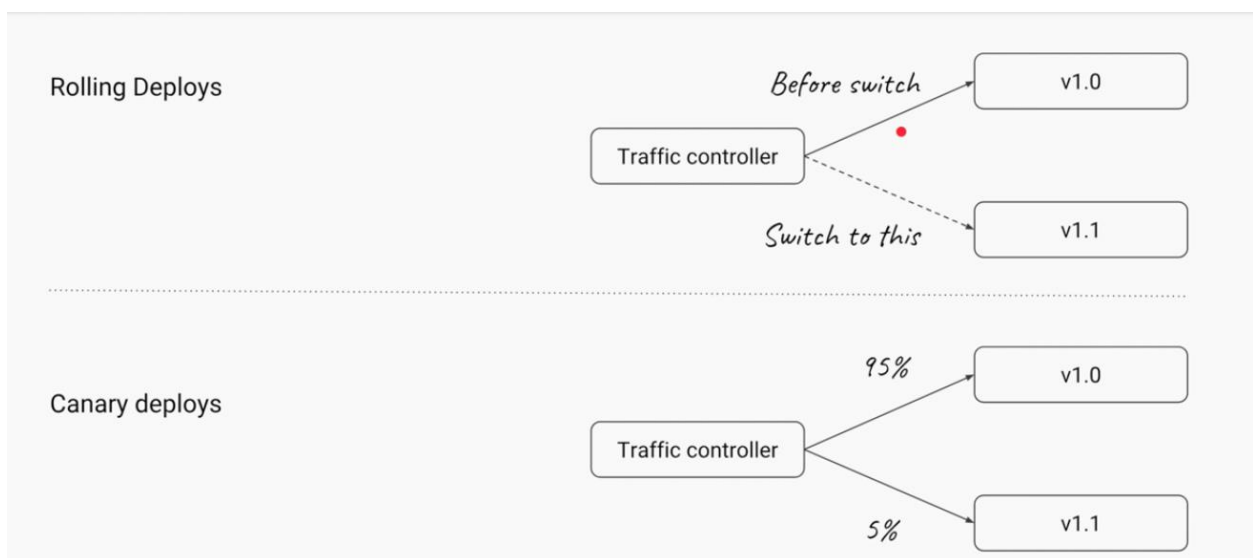




Load Balancing

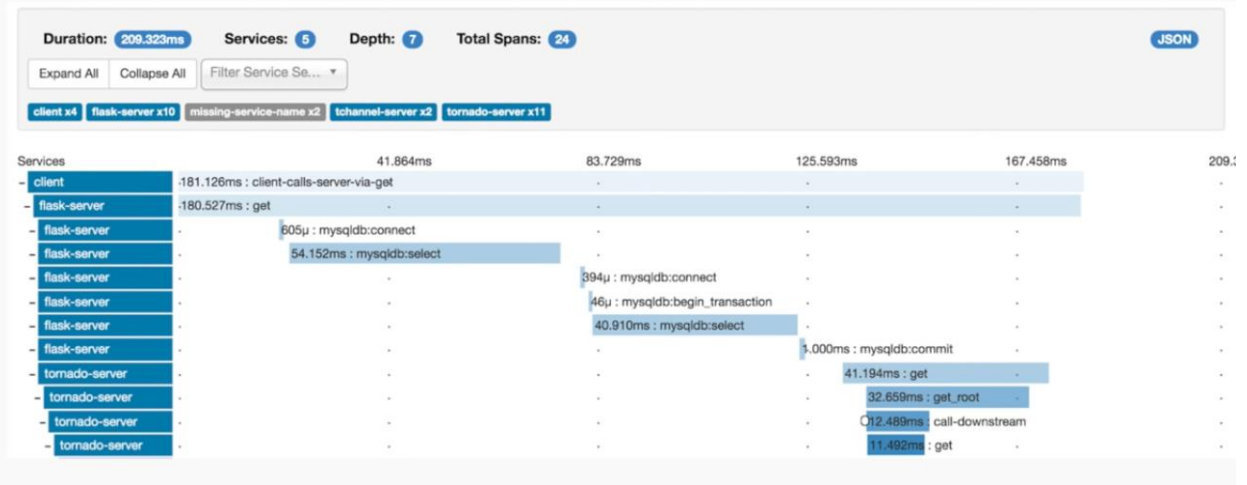


Canary/Rolling Deploys



Distribution Tracking

- Spring Cloud Sleuth
- Adding trace-id, span-id, parent-span-id



Security

- Mutual TLS between services
- Network policies / Whitelisting
- Transparent patching

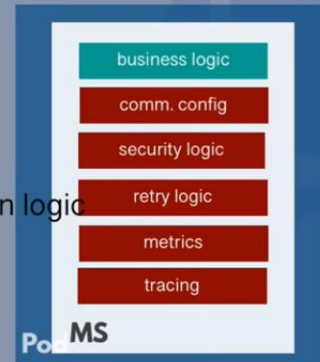
Challenges of a microservice architecture



✗ These **NON business logic** must be added to **each application**

✗ Developers don't work on actual service/application logic

✗ adds complexity to the services



Service Mesh

Motto

Do not burden my code with all these infrastructure related decisions

Solution: Service Mesh with Sidecar Pattern

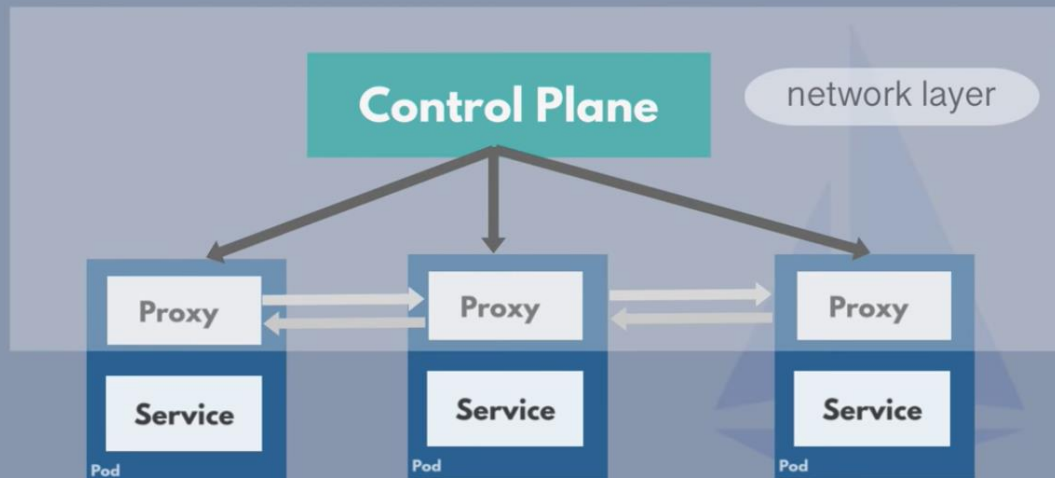
Sidecar Proxy

- ▶ handles these networking logic
- ▶ acts as a **Proxy**
- ▶ third-party application
- ▶ cluster operators can configure it easily

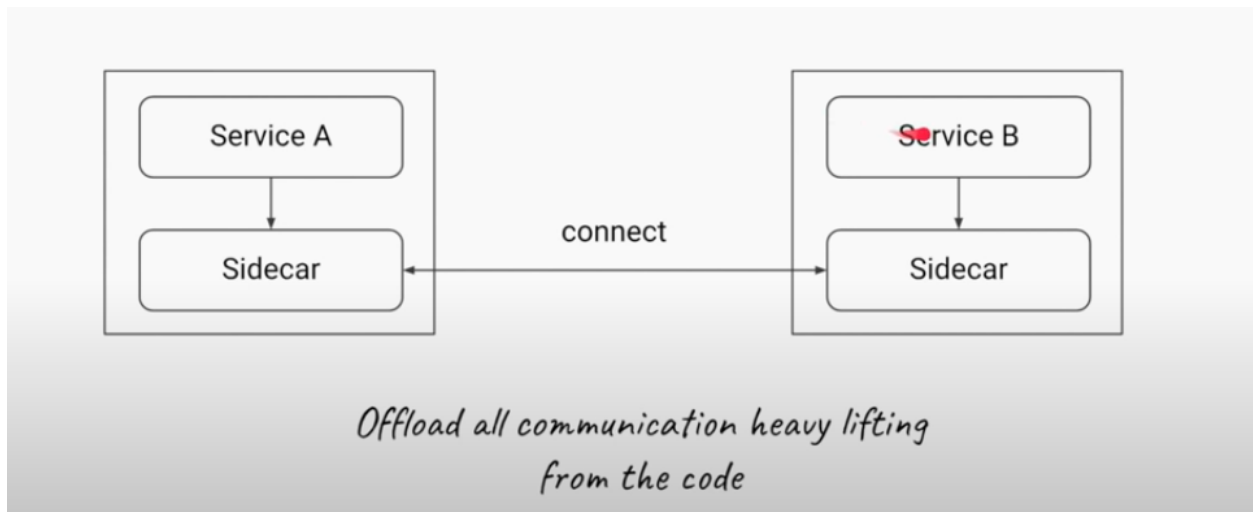
Control Plane



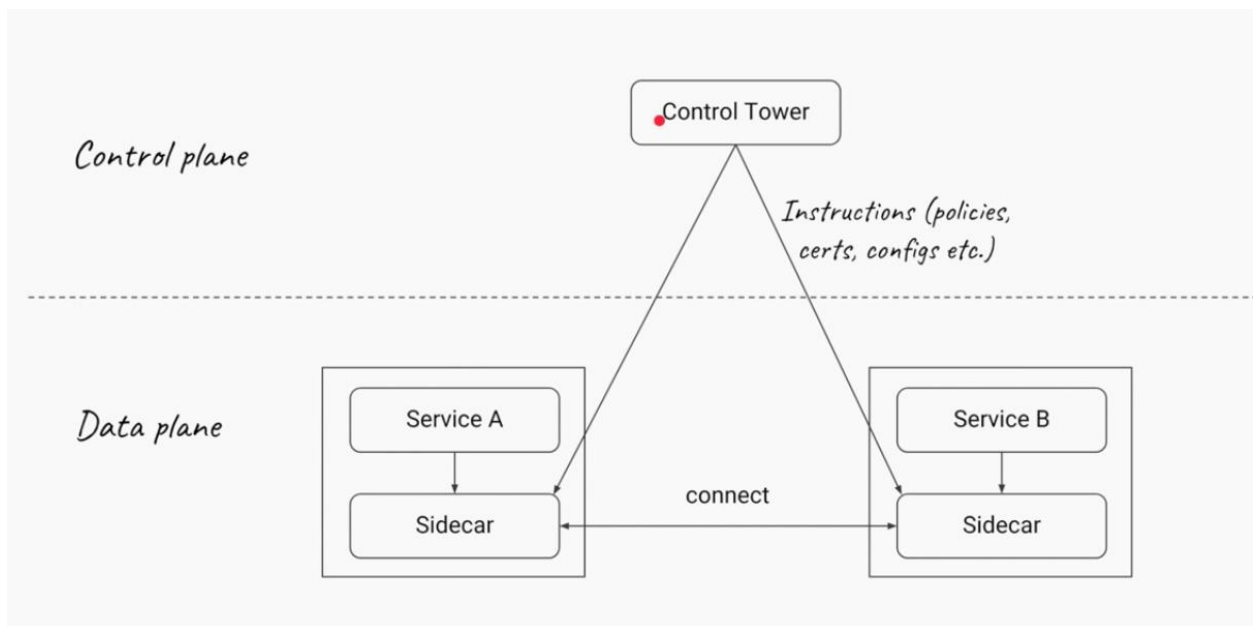
Solution: Service Mesh with Sidecar Pattern

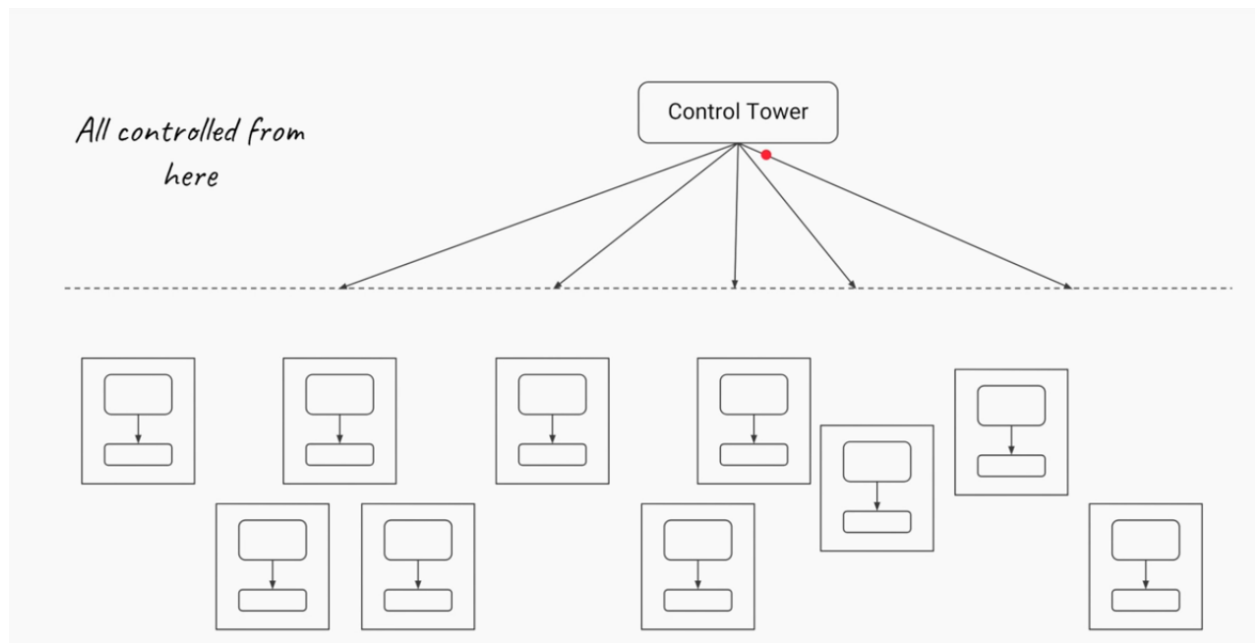
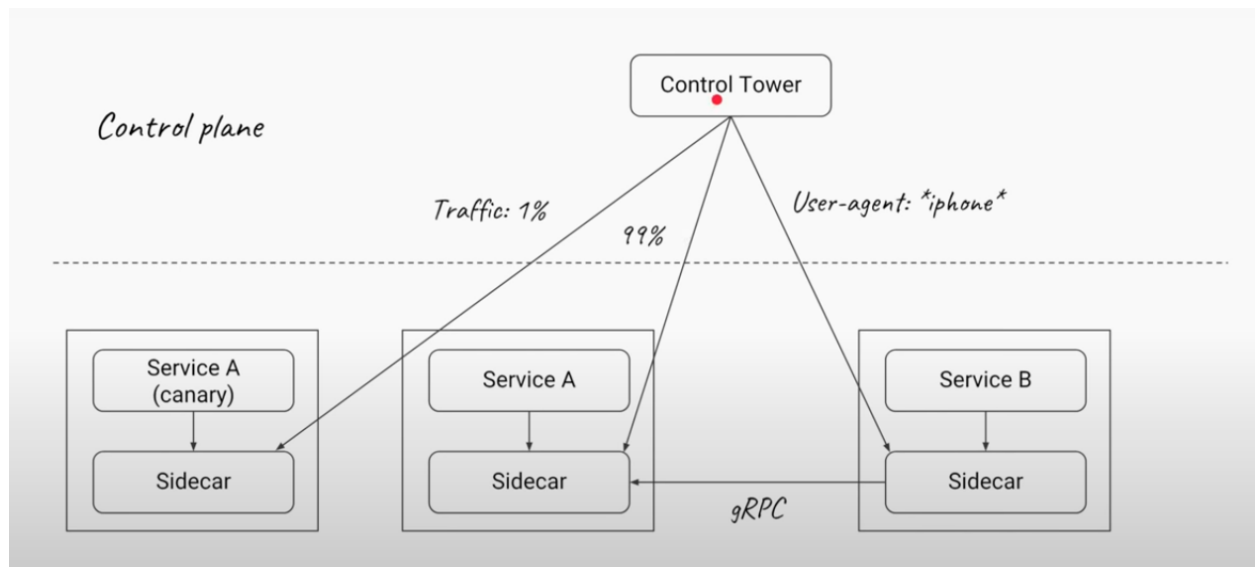


Sidecar

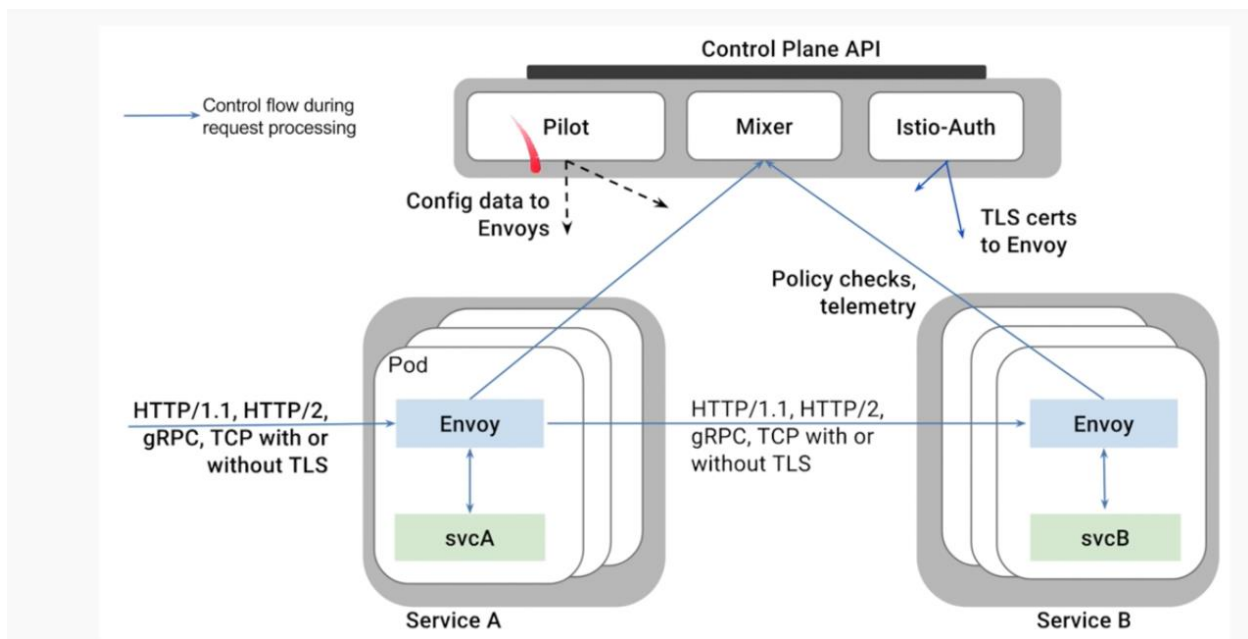


Data Plane Vs Control Plane





Istio + Envoy



Istio is a Service Mesh, Service Mesh manages communication between microservices

Core feature: Traffic Splitting

 **Payment Service**

Service Version 3.0

- ▶ deployed to **production**
- ▶ you can't be sure that there is NO bug



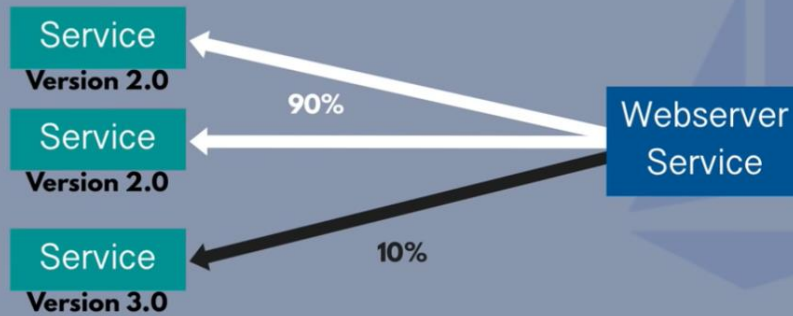
Core feature: Traffic Splitting



Payment Service



release new versions without worrying about breaking the application



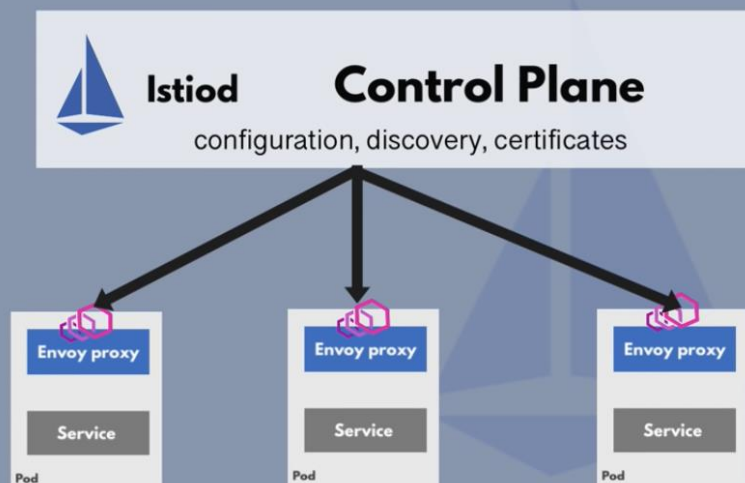
Service Mesh is a Pattern or Paradigm



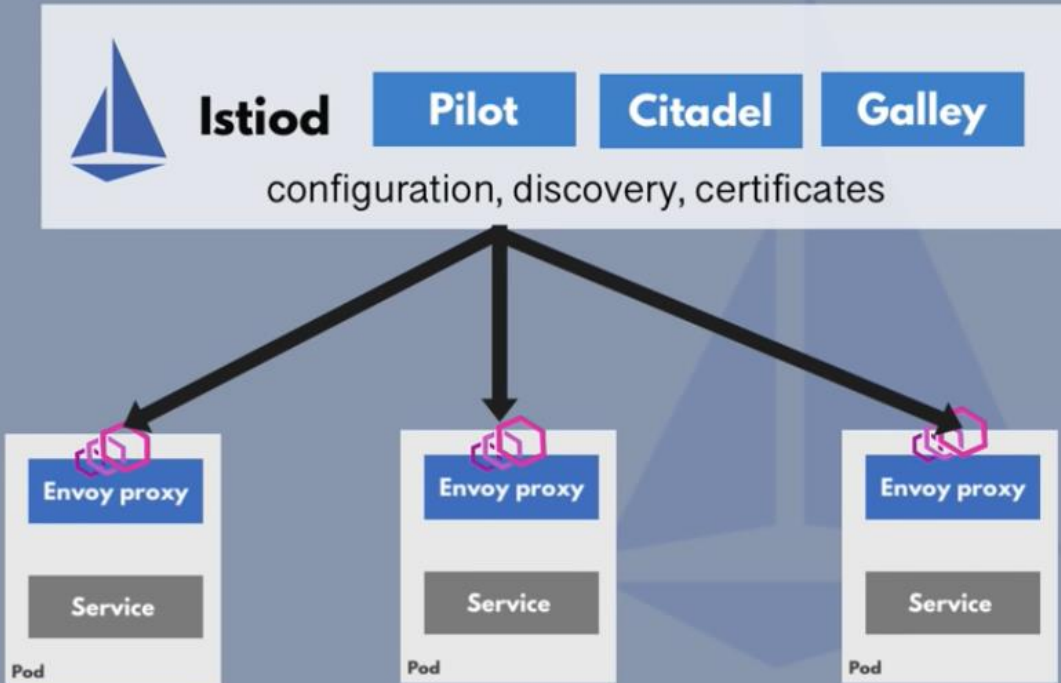
Istio

Istio is an **implementation**

Istio Architecture



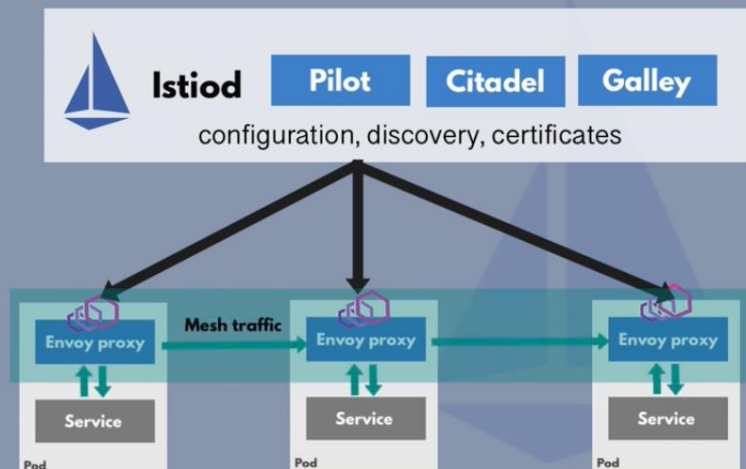
Istio Architecture



Istio Architecture

Control Plane

Data Plane



How to configure Istio?



Istio is configured with **Kubernetes** **YAML** files



- Istio uses Kubernetes CustomResourceDefinitions (CRD)

CRD

```
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: payment-destination
spec:
  host: payment.prod.svc.cluster.local
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
```

How to configure Istio?



Istio is configured with **Kubernetes** **YAML** files



- Istio uses Kubernetes CustomResourceDefinitions (CRD)

CRD

- extending the Kubernetes API
- **custom** Kubernetes component/object for e.g. third-party technologies (like Istio, Prometheus, etc.)
- can be used like any other native Kubernetes objects

How to configure Istio?



Istio is configured with **Kubernetes YAML files**

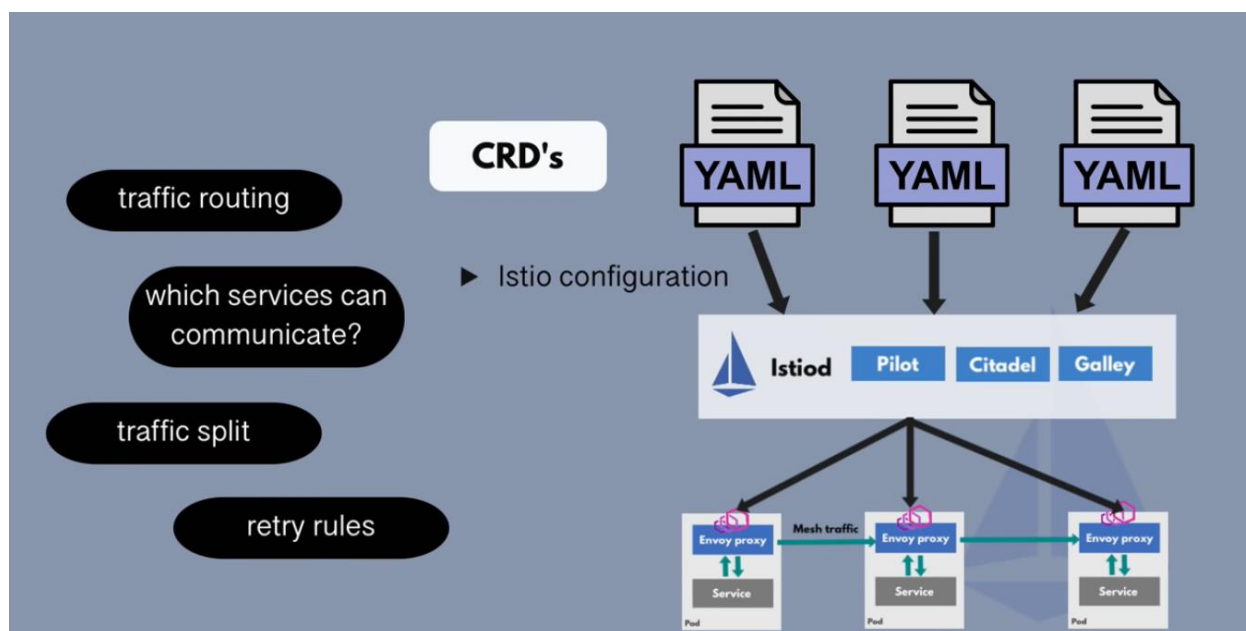


no need to learn Istio specific language



CRD

- ▶ extending the Kubernetes API
- ▶ **custom** Kubernetes component/object for e.g. third-party technologies (like Istio, Prometheus, etc.)
- ▶ can be used like any other native Kubernetes objects



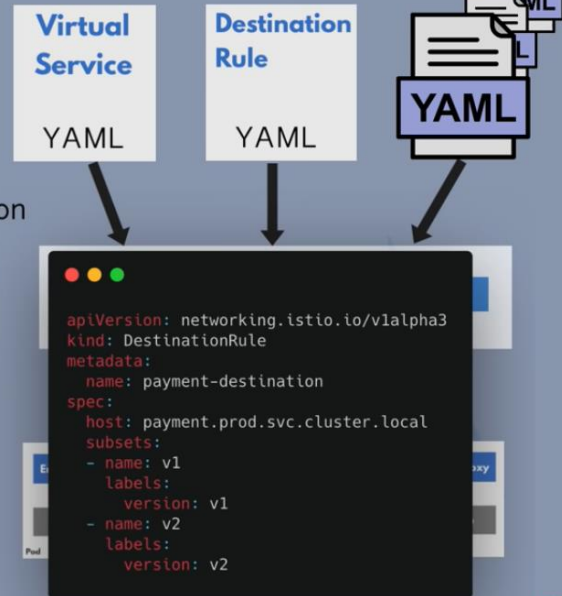
Key building blocks of Istio's traffic routing functionality

► **VirtualService:**

How you route your traffic **TO** a given destination

► **DestinationRule:**

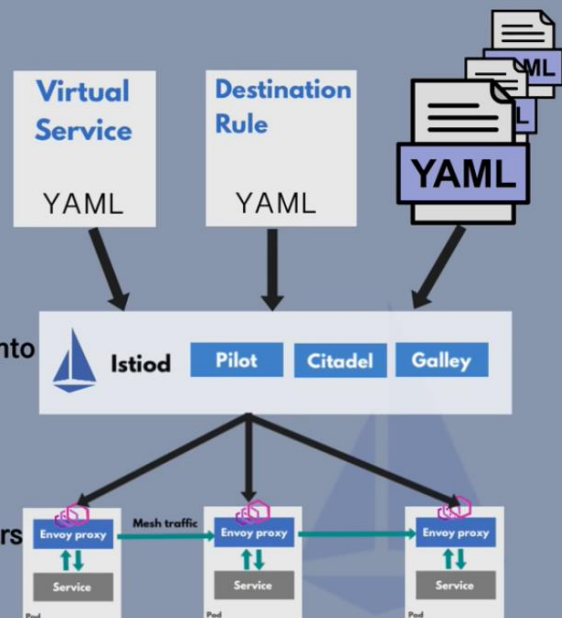
Configure what happens to traffic **FOR** that destination



► We create CRD's

► Istiod **converts** these high level routing rules into Envoy-specific configurations

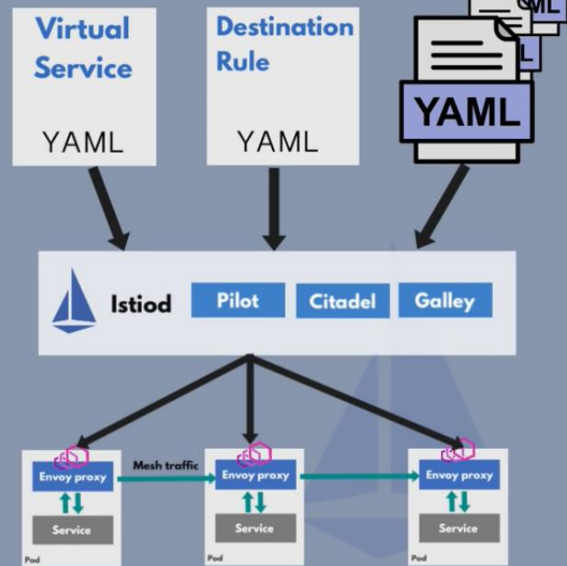
► Configuration is **propagated** into Proxy sidecars



We don't configure Proxies,
we configure Istiod



► Proxies can communicate without
connecting to Istiod



Dynamic Service discovery

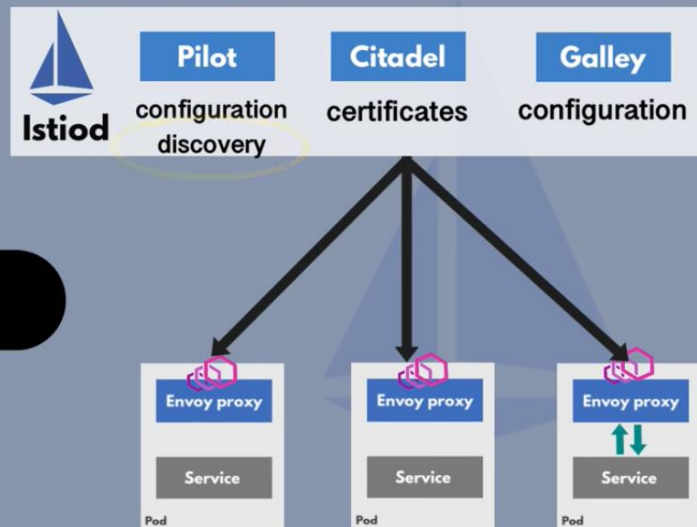
Istio

- ✓ configuration
- ✓ service discovery

Internal Registry for Services
& their endpoints



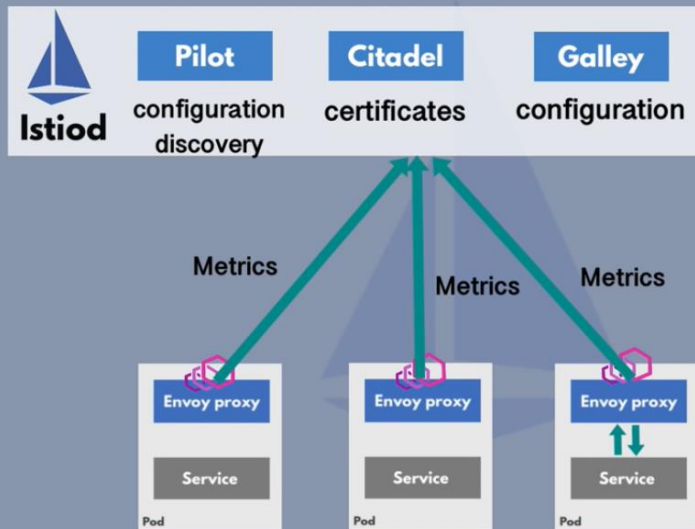
no static configuration
of endpoints



Metrics and Tracing

Istio

- ✓ configuration
- ✓ service discovery
- ✓ certificate management
- ✓ gather telemetry data

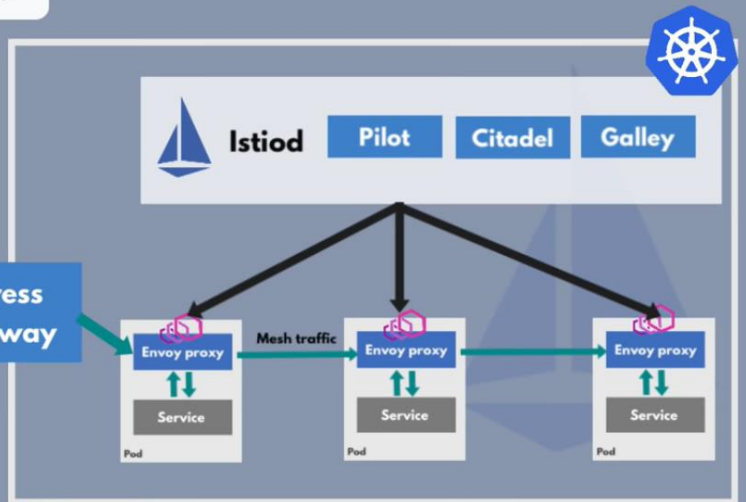


Istio Ingress Gateway

entrypoint to your cluster

Cluster Request

Ingress Gateway



Istio Ingress Gateway

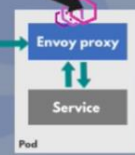
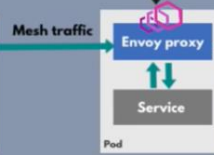
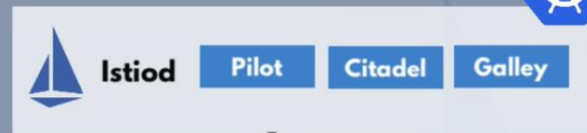
entrypoint to your cluster

- Gateway directs traffic to MS using VirtualService

Cluster Request

Ingress Gateway

Virtual Service



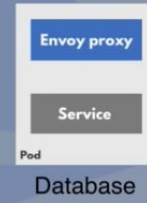
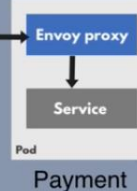
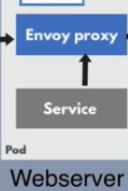
Istio: Traffic Flow



Request

Istio Gateway

Virtual Service



mTLS

mTLS

Istio: Traffic Flow

